

61. XMLHttpRequest (XHR)

Browser ka built-in object jo JS me HTTP requests karta hai. Naam me "XML" hai, par ye **kisi bhi data** par kaam karta hai (files upload/download, progress track karna, etc.).

Aajkal `fetch` modern hai aur XHR ko kaafi had tak deprecate karta hai. Phir bhi XHR 3 wajah se use hota hai:

1. **Historical:** purani scripts support karni hain.
2. Purane browsers chahiye aur polyfill nahi chahiye.
3. Kuch aisa chahiye jo `fetch` abhi nahi kar sakta, jaise **upload progress**.

Basics (asynchronous mode)

4 steps:

```
js
let xhr = new XMLHttpRequest(); // 1. banao
xhr.open('GET', '/article/xmlhttprequest/example/load'); // 2. configure
xhr.send(); // 3. bhejo
xhr.onload = function() { ... }; // 4. events suno
```

`xhr.open(method, URL, [async, user, password])`

- `method`: "GET", "POST" etc.
- `URL`: string ya `URL` object
- `async`: `false` do to synchronous request
- `user, password`: basic HTTP auth

`open` naam ke bawajood **connection nahi kholta**, sirf configure karta hai. Network activity `send` se shuru.

`xhr.send([body])`: `GET` me body nahi hoti, `POST` me data body me jaata hai.

Sabse zyaada use hone wale 3 events

- `load`: request पूरी ho gayi (**400/500 status par bhi**) aur response fully download
- `error`: request hi nahi ho payi (network down, invalid URL)
- `progress`: response download hote waqt baar baar (`event.loaded`, `event.total`, `event.lengthComputable`)

```
js
```

```

xhr.onload = function() {
  if (xhr.status != 200) alert(`Error ${xhr.status}: ${xhr.statusText}`);
  else alert(`Done, got ${xhr.response.length} bytes`);
};

xhr.onprogress = function(event) {
  if (event.lengthComputable) alert(`Received ${event.loaded} of ${event.total}`);
  else alert(`Received ${event.loaded} bytes`); // Content-Length nahi
};

xhr.onerror = function() { alert("Request failed"); };

```

Response ki properties

- `status`: HTTP code (200, 404...), non-HTTP failure par `@`
- `statusText`: `OK`, `Not Found`...
- `response` (purani scripts `responseText` use karti hain): body

Timeout

```

js
xhr.timeout = 10000; // ms

```

Is time me na ho to cancel aur `timeout` event.

URL parameters

`URL` object se properly encode:

```

js
let url = new URL('https://google.com/search');
url.searchParams.set('q', 'test me!');
xhr.open('GET', url);

```

`responseType`

Response ka format:

- `""` / `"text"`: string
- `"arraybuffer"`: `ArrayBuffer`
- `"blob"`: `Blob`
- `"document"`: XML/HTML document
- `"json"`: automatically parse hua JSON

js

```
xhr.open('GET', '/article/xmlhttprequest/example/json');
xhr.responseType = 'json';
xhr.send();
xhr.onload = () => alert(xhr.response.message);
```

Purani scripts me `responseText`, `responseXML` milte hain (historical). Ab `responseType` + `response`.

Ready states (`xhr.readyState`)

```
UNSENT = 0           // shuruaati
OPENED = 1           // open call hua
HEADERS_RECEIVED = 2 // response headers aaye
LOADING = 3          // response load ho raha hai (har packet par repeat)
DONE = 4              // poora
```

Order: `0 -> 1 -> 2 -> 3 -> ... -> 3 -> 4`. Track karne ke liye `readystatechange` event, jo purane code me milta hai. Ab `load/error/progress` use karo.

Request abort karna

js

```
xhr.abort(); // 'abort' event aata hai, status = 0
```

Synchronous requests

`open` ka teesra parameter `false` ho to `send()` par JS **ruk jaata hai** jab tak response na aaye (`alert` jaisa). Kaafi kam use hota hai kyunki page block ho jaata hai, aur timeout, cross-domain, progress jaise features nahi milte. `onerror` ki jagah `try..catch`.

HTTP headers

- `setRequestHeader(name, value)`: header set karo. Kuch headers (`Referer`, `Host`) sirf browser set karta hai.
 - Header hata nahi sakte**: baar baar call karne par value **jud jaati hai** (`X-Auth: 123, 456`).
- `getResponseHeader(name)`: ek response header (`Set-Cookie` chhodkar)
- `getAllResponseHeaders()`: saare headers ek line me (`\r\n`) se alag, `:` se name/value). Object banane ke liye split + reduce.

POST aur FormData

js

```
let formData = new FormData(document.forms.person);
formData.append("middle", "Lee");

let xhr = new XMLHttpRequest();
xhr.open("POST", "/article/xmlhttprequest/post/user");
xhr.send(formData); // multipart/form-data
xhr.onload = () => alert(xhr.response);
```

JSON bhejna ho to:

js

```
xhr.open("POST", '/submit');
xhr.setRequestHeader('Content-type', 'application/json; charset=utf-8');
xhr.send(JSON.stringify({ name: "John", surname: "Smith" }));
```

`send(body)` lagbhag sab kuch leta hai, `Blob` aur `BufferSource` bhi.

Upload progress

`xhr.onprogress` sirf **download** stage par chalta hai. Upload track karne ke liye alag object:

`xhr.upload` (sirf events, methods nahi):

- `loadstart`, `progress`, `abort`, `error`, `load`, `timeout`, `loadend`

js

```
function upload(file) {
  let xhr = new XMLHttpRequest();

  xhr.upload.onprogress = function(event) {
    console.log(`Uploaded ${event.loaded} of ${event.total}`);
  };

  xhr.onloadend = function() {
    if (xhr.status == 200) console.log("success");
    else console.log("error " + this.status);
  };

  xhr.open("POST", "/article/xmlhttprequest/post/upload");
  xhr.send(file);
}
```

Cross-origin requests

`fetch` jaisi hi CORS policy. Default me cookies/HTTP-authorization nahi jaate. Bhejne ke liye:

```
js

xhr.withCredentials = true;
```

Summary: sabhi events (lifecycle order me)

- `loadstart`: request shuru
- `progress`: response ka data packet aaya
- `abort`: `xhr.abort()` se cancel
- `error`: connection error (HTTP 404 jaisi error par nahi)
- `load`: successfully khatam
- `timeout`: timeout se cancel
- `loadend`: `load`, `error`, `timeout` ya `abort` ke baad

`error`, `abort`, `timeout`, `load` me se **sirf ek** hota hai. Upload track karna ho to yahi events `xhr.upload` par.